AI Documentation Generator

Agentic AI Project — Step by Step Workflow

This document explains, in simple terms, the complete step-by-step process the system follows — from the moment a developer submits their project, to the moment they receive a finished, ready-to-use documentation file.

Overview

In one line: **Input** → **Unpack Project** → **Understand Code** → **Plan Sections** → **Write Sections (parallel)** → **Check & Correct** → **Write Summary** → **Assemble** → **Deliver.**

Step 1: User Provides Input

The user either pastes a Git repository URL, or uploads a Zip file of their project.

Step 2: Cloning / Unpacking the Project (Ingestion)

If it's a Git URL, the system clones the repository. If it's a Zip file, it extracts it. Unnecessary stuff like **node_modules**, **.git**, and build files are cleaned up.

This step builds a basic map of the project: the file tree, file types, and overall size.

Step 3: Making Sense of What's Inside (After Cloning)

Once the repo is cloned locally, all we have is a folder full of raw files. The system doesn't understand anything about the project yet, so before writing any documentation, we need to build a clear picture of the project, step by step:

- **Look at the folder structure first** — scan what files and folders exist, ignoring junk folders like node_modules, .git, dist, build. This gives a clean map of the project.
- **Figure out what kind of project it is** — look for clue files such as package.json (JavaScript/Node), requirements.txt (Python), or pom.xml (Java). This tells us the language and framework without even reading the actual code yet.
- **Find the entry point of the project** — every project has a starting file, like main.py, index.js, or app.py. Finding this tells us how the project actually runs.

Step 4: Read the Code, File by File, and Summarize It

After finding the entry point, the next step is to actually start reading and understanding the code itself — but smartly, not all at once. Instead of dumping the entire codebase into the AI at once (too much and messy), we go file by file and ask a simple question for each one: **'What does this file do?'**

So for every important file, a short summary is generated, for example:

- "This file handles user login and authentication"
- "This file defines the database models for orders and customers"
- "This file sets up the API routes for the product page"

This turns hundreds of files into a much smaller, easy-to-understand set of summaries.

Step 5: Figure Out How Files Are Connected to Each Other

Once we know what each file does, the next thing is to understand how they talk to each other. For example: does app.py import and use auth.py? Does orders.py depend on database.py?

This builds something like a 'map of connections' between files — this is what eventually helps write the **System Architecture** section later, because architecture is really just 'how the pieces connect.'

Step 6: Detect Special Things in the Project

While going through the code, we also check for specific signals:

- Is there a database? (look for ORM models, SQL files, DB config)
- Is there any AI/ML code? (look for libraries like TensorFlow, PyTorch, model files)
- Is there an API? (look for routes/controllers)

These flags matter because they decide which optional sections (like Database Design or AI/ML Models) actually need to be written later.

Summary so far: after finding the entry point, the job is not to write documentation yet — it's to read the code piece by piece, summarize each part, understand how the pieces connect, and flag any special features (DB, ML, API). All of this together becomes the project's 'understanding,' which gets handed off to the next stage — the agent that decides which documentation sections to actually generate.

Step 7: Decide Which Sections Actually Apply to This Project

The final document has 14 possible sections, but not every project needs all of them. This step looks at everything learned in the previous steps and makes decisions like:

- "This project has a database → include the Database Design section"
- "This project has no AI/ML code → skip the AI/ML Models section"
- "This project has API routes → definitely include API Documentation"

Basically, it's building a checklist: 'these are the sections we need to write for this particular project.'

Step 8: Decide the Order and Dependencies for Writing Sections

Some sections can be written independently and don't depend on anything else — like Tech Stack, Folder Structure, Installation Guide. These can all be done at the same time, since they don't need each other's content.

But some sections depend on others being finished first — for example, the **Abstract** can only be written properly after the Problem Statement, Objectives, and Features sections are done, because it summarizes them.

So this step also decides which sections can be written in parallel, and which ones need to wait.

Step 9: Each Writing Agent Drafts Its Own Section

Now that the plan is ready, multiple specialized 'writer' agents kick in — and importantly, many of them work at the same time, not one after another. This makes the whole process much faster.

Each writer only looks at the small piece of project understanding that's relevant to it. For example:

- The agent writing **Installation Guide** only looks at things like requirements.txt, package.json, or Docker files.
- The agent writing **API Documentation** only looks at the routes/endpoints found earlier.
- The agent writing **Database Design** only looks at the database models/schema.

This keeps each writer focused and accurate, instead of one agent trying to juggle the entire project at once and mixing things up. At the end of this step, we have draft content for every applicable section — but it's not final yet, because nothing has been checked for accuracy.

Step 10: The Verifier Checks Every Drafted Section Against the Real Code

Once a section is written, we don't just trust it blindly — AI can sometimes get things wrong or 'make up' details that sound right but aren't actually true. So this step is like a quality control check. For each section, the Verifier asks: 'Does this actually match what's really in the project?'

- If the Installation Guide says "run npm install," the Verifier checks — does this project actually have a package.json? If not, that's wrong.
- If the API Documentation lists an endpoint like /users/login, the Verifier checks — does that endpoint actually exist in the code?
- If the Database Design section describes a "Customers" table, the Verifier checks — does that table actually exist in the project's database files?

Step 11: If Something's Wrong, It Goes Back to Be Fixed

If the Verifier finds a mistake, it doesn't just reject the whole thing — it sends that specific section back to the writer agent that created it, along with a note about what was wrong. That writer then redoes just that one section, and the Verifier checks it again.

This repeats a few times until it's correct. If it still isn't right after a few tries, the system flags it for a human to review manually instead of guessing further.

If everything checks out, that section is marked as 'done and verified.'

Step 12: Writing the Abstract and Table of Contents

Now that all the individual sections are written and confirmed correct, one last writer agent steps in to create the two sections that go at the very top of the document:

- **Abstract** — a short, high-level summary of the whole project. It's written last on purpose, because it needs to accurately reflect the Problem Statement, Objectives, and Features — and those are only trustworthy now that they've been checked.
- **Table of Contents** — simply lists out the final sections that are actually included (remember, some sections like Database Design or AI/ML Models might have been skipped if they didn't apply), with proper numbering.

This step is quick and mostly summarizing work, not new research.

Step 13: Putting Everything Together Into One Final Document

At this point, we have all the individual pieces — Abstract, Overview, Problem Statement, Features, Architecture, Tech Stack, Installation Guide, and so on — each one already checked for correctness.

This last step is mostly just formatting and assembling, not really 'thinking' work:

- Arrange all sections in the correct order (1 to 14, skipping any that weren't needed)
- Make headings, spacing, and formatting consistent throughout
- Add proper links in the Table of Contents
- Export it into the final file format (Markdown, PDF, or Word doc — whatever the user wants)

Step 14: Deliver the Final Document to the User

The finished, polished documentation is handed back to the developer who submitted their project in the first place. Optionally, before finalizing, the user could preview it and request changes — but the core pipeline work is done at this point.

Final Summary

Clone/Unzip → Understand the code → Decide what sections are needed → Write each section (in parallel) → Check each section for accuracy (fix if wrong) → Write the summary once everything's verified → Assemble the final document → Deliver it to the user.

That's the complete pipeline from start to finish.